

Implementazione del Metodo delle Correnti di Maglia per la risoluzione di circuiti elettrici

Corso di Programmazione e Calcolo Scientifico

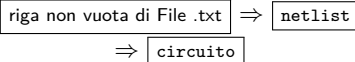
Noemi Fantini (327396), Francesca Gregorio (322444), Mariarosa Labella
(322015)

Politecnico di Torino

2026-17-06

Architettura e Performance

- **Separazione delle responsabilità:**



- **Mappe associative:** per legare la geometria del circuito a proprietà fisiche.

edge $\Rightarrow$ segno

Risoluzione delle criticità

- **Orientamento degli archi:** mappa segno basata sul confronto lessicografico dei nodi:

```
undirected_edge<int> e_p(1,2);  
C.segno[e_p] = +1;  
undirected_edge<int> e_m(2,1);  
C.segno[e_m] = -1;
```
- **Uno sguardo al futuro:** suddivisione in base a `nl.tipo()`:

```
nl.tipo() == 'R'  $\Rightarrow$  C.R.push_back(nl);
```

```
nl.tipo() == 'V'  $\Rightarrow$  C.V[e]=nl.valore();
```

```
con undirected_edge<int>  
e(nl.nodo1(), nl.nodo2());
```

Meccanismo di Parsing

- **Tokenizzazione:** lettura riga per riga (`std::getline`) e scomposizione robusta tramite `std::istringstream`.

"R1 10 1 2" → name, val, n1, n2

- **Validazione:** verifica del tipo e parsing dei dati.

Esempi da `netlist_esempio.txt`:

R1 10 1 2 → tipo: 'R' | nome: "R1" | valore: 10.0 | nodo1: 1 | nodo2: 2

→ La riga vuota viene ignorata

V2 20 4 3 → tipo: 'V' | nome: "V2" | valore: 20.0 | nodo1: 4 | nodo2: 3

Gli spazi aggiuntivi vengono ignorati

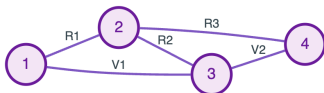
Logica Anti-Errore

- **Gestione degli input validi:** gestione di file mancanti o parametri extra tramite `std::optional`.
- **Vincoli fisici:** uso di `std::set` per scartare le resistenze in parallelo.
- **Robustezza:** ottenuta ignorando spazi extra e righe vuote.
- **Stato invalido:** se il parsing fallisce, la riga viene saltata senza interrompere l'esecuzione.

Scelta del modello: grafo indiretto e non pesato

La geometria di C.G è isolata dalle grandezze fisiche: l'arco rappresenta solo la connessione. Segno, valori e indici sono gestiti esternamente dalla struttura circuito.

Grafi e rappresentazione in memoria



L'ordinamento del grafo

L'ordinamento del grafo è un invariante garantito ad ogni inserimento di archi:

- **Ricerca della posizione:** il punto corretto viene individuato tramite `std::lower_bound`. Ciò permette di mantenere l'ordinamento.

```
std::vector<std::vector<T>> m_adj
```

Nodo [0]	Vicini [1...n]
1	2, 3
2	1, 3, 4
3	1, 2, 4
4	2, 3

L'ordinamento dei vicini elimina la necessità di sort finali.

Output di C.G.all_edges():

{(1, 2), (1, 3), (2, 3), (2, 4), (3, 4)}

I duplicati sono gestiti tramite la condizione ($u \leq v$).

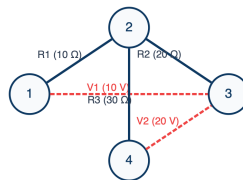
Idea

- Ogni arco del coalbero genera un ciclo fondamentale.
- `findpath` ricostruisce il cammino nel DFS tree.
- Uso di `visited` per tracciare i nodi visitati.
- Chiusura del ciclo.

Assunzioni

- Grafo connesso.
- Gestione del backtracking in `findpath`.

Esempio:



Output:

Ciclo n. 1:

1 2 3 1

Ciclo n. 2:

2 3 4 2

Scelte implementative

- Linearizzazione con **Dijkstra**
- Operazioni bitwise implementate con le operazioni logiche **&** e **XOR**.

Criticità

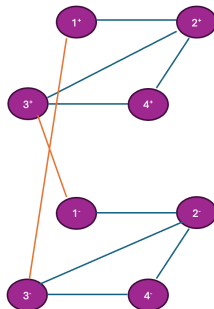
- **Cicli degeneri e cappi** $\Rightarrow$ controllo dimensionale nella conversione finale.
- **Nodi duplicati** $\Rightarrow$ lifted_node<T>
- **Incompatibilità dimensionali** $\Rightarrow$ validazione preventiva sulla dimensione globale degli archi.

Esempio netlist_esempio.txt



Continua Esempio

G_prime, vettore di incidenza relativo a e(1,3)



Output, base dei cicli minimi:

Ciclo n. 1:

1 1 1 0 0

Ciclo n. 2:

0 0 1 1 1

La funzione `bool_cycle_to_nodes` trasforma il ciclo ottenuto (vettore di incidenza booleano) in una sequenza ordinata di nodi.

Criticità e soluzioni

- **Assenza di ordinamento:** il vettore booleano identifica gli archi del ciclo, ma non in ordine $\Rightarrow$ costruzione di una mappa di adiacenza.
- **Backtracking e cicli degeneri:** tornare sull'arco appena percorso o chiudere un falso ciclo di due nodi $\Rightarrow$ tracciare il nodo precedente in `prev`, usare il flag `primo_passo` e imposizione di una condizione per la chiusura del ciclo (`path.size() >= 3`).
- **Vicoli ciechi:** una scelta errata del nodo successivo $\Rightarrow$ i cicli prodotti da De Pina garantiscono che ogni nodo nel sottografo degli archi attivi abbia grado esattamente 2.

Esempio `netlist_esempio.txt`

Output, base dei cicli minimi:

Ciclo n. 1:

1 1 1 0 0

Ciclo n. 2:

0 0 1 1 1

Ricostruzione dei cicli:

Ciclo n. 1:

1 2 3 1

Ciclo n. 2:

2 3 4 2

Riempimento della matrice

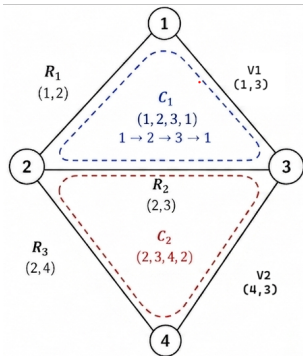
Per ogni arco appartenente a un ciclo:

- si verifica la corrispondenza con una resistenza del circuito
- in caso di match si assegna:

$$B_{ij} = \begin{cases} +1 & \text{se } u < v \\ -1 & \text{se } u > v \end{cases}$$

Scelte implementative

- Matching tramite confronto di `undirected_edge`
- Ricerca lineare sulle resistenze ($O(m)$)
- break dopo il match (unicità arco)



$$B = \begin{bmatrix} +1 & 0 \\ +1 & -1 \\ 0 & +1 \end{bmatrix}$$

Struttura e riempimento del vettore

Per ogni maglia:

- si percorrono gli archi del ciclo;
- se l'arco contiene un generatore di tensione:

$$v_j \ += \text{segno} \cdot \text{valore}$$

Problema

Conciliare:

- Polarità del generatore
- Verso di percorrenza della maglia

$$v = \begin{bmatrix} 10 \\ -20 \end{bmatrix}$$

Soluzione implementata

$$\text{segno_complessivo} = \text{segno_gen} \cdot \text{segno_maglia}$$

- `segno_gen`: polarità reale
- `segno_maglia`: verso del ciclo

$$v[j] \ += (\text{segno_gen} \cdot \text{segno_maglia}) C.V[e_j]$$

Struttura del sistema

$$\underbrace{B^T R B}_A i = v$$

$$A = B^T R B$$



A SPD (simmetrica definita positiva)

Criteri di arresto

Accuratezza $\leftrightarrow$ Tempo di calcolo $\leftrightarrow$ Convergenza

- **Tolleranza relativa:** 10^{-12} , controllo sul residuo relativo $\rightarrow$ accuratezza della soluzione
- **Iterazioni max:** $10 \cdot n$, criterio di sicurezza rispetto al condizionamento del sistema

Metodo risolutivo

Gradiente Coniugato

- **Efficienza computazionale:** solo prodotti matrice-vettore, nessuna inversione o fattorizzazione
- **Scalabilità:** adatto a sistemi di grandi dimensioni

Una volta ottenute le correnti di maglia, si ricostruiscono le grandezze elettriche del circuito.

Ricostruzione delle grandezze fisiche

- **Correnti nei resistori:**

$$I_{res} = BI$$

- **Tensioni ai capi dei resistori:**

$$V = R (BI)$$

Output del programma

Per ogni resistore vengono stampati:

- nome del componente
- tensione V
- corrente I

Esempio: netlist_esempio.txt

```
R1:  V = 0.91 volts, I = 0.09 amps.  
R2:  V = 9.09 volts, I = 0.45 amps.  
R3:  V = -10.91 volts, I = -0.36 amps.
```